

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Gabriela Pricop, FAF-241

Report

Laboratory Work No. 3

Operating Systems

Checked by:
Reițman Patricia, *university assistant*

Chișinău – 2026

Contents

1 Part 1: Move fast, find help **3**

2 Part 2: What the shell does to your line **6**

3 Part 3: Redirection, chaining and jobs **10**

4 Part 4: Answer questions with one-liners **13**

5 Part 5: Your first script **17**

Part 1: Move fast, find help

Questions:

- Which of cd, ls, echo are built-ins, and why MUST cd be one?
- How many directories are in your PATH, and in which one is ls?
- What exit code does a command that is not found return?
- Which option of ls sorts by modification time? (find it in man ls)

Answers:

- `cd` and `echo` are shell built-ins, while `ls` is an external program (`/usr/bin/ls`, which is also aliased to `ls -color=auto`). `cd` must be a built-in because the working directory belongs to the shell process itself. If `cd` were an external program, it would run in a child process, change the directory of that child only, and then exit, leaving the current directory of the shell unchanged.
- The PATH contains 10 entries (one of them, `/snap/bin`, appears twice). The command `ls` is located in `/usr/bin`.
- A command that is not found returns exit code 127.
- The option `-t` of `ls` sorts the files by modification time, newest first (found in `man ls`).



Figure 1.1: ls /usr/sh before pressing tab



Figure 1.2: ls /usr/sh after pressing tab



Figure 1.3: cd command before pressing tab



Figure 1.4: cd after tab completion

ls /usr/sh + Tab Tab, cd /et + Tab – the Tab key asks the shell to complete the word under the cursor. Pressing Tab twice after `ls /usr/sh` makes the shell list all the entries of `/usr` that start with `sh`, because there is more than one possible completion. Pressing Tab once after `cd /et` completes the name to `/etc/`, because only one entry matches. This saves typing and prevents mistakes in long paths.

Ctrl+R – starts a reverse search through the command history. After running three different commands, pressing Ctrl+R and typing a few letters of the first one makes the shell show the most recent command that contains those letters, and Enter runs it again. This is the fastest way to repeat a long command without retyping it.

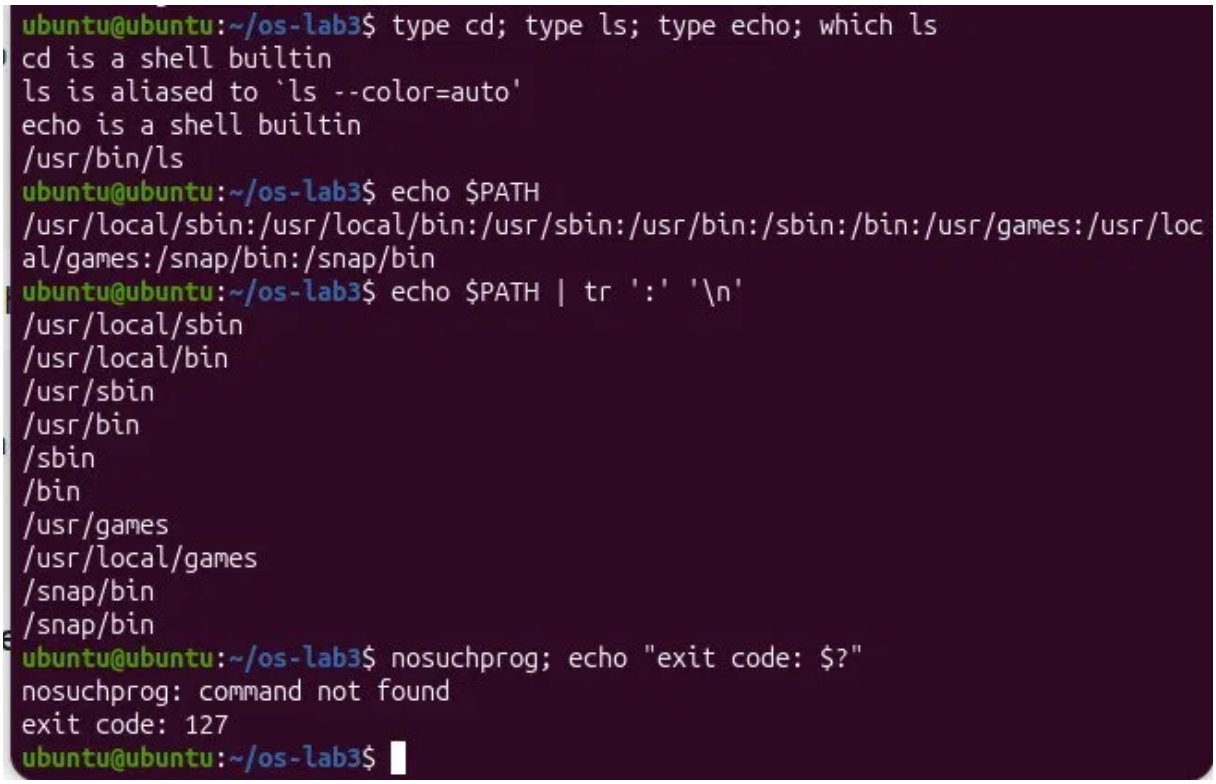


Figure 1.5: Output of type, which, PATH and the exit code of a missing command

type cd ; type ls ; type echo ; which ls – `type` tells how the shell interprets each name, and `which` prints the path of an external executable. As seen in Figure 1.5, `cd` and `echo` are reported as shell built-ins, `ls` is reported as an alias of `ls -color=auto`, and `which ls` prints `/usr/bin/ls`. This shows that built-ins are executed inside the shell, while external programs are files found on the disk.

echo $PATH – prints the value of the `PATH` variable, which is a list of directories separated by colons. The shell searches these directories, from left to right, when a command name is typed without a slash.

echo $PATH | tr ':' '\n' – the `tr` filter replaces every colon with a newline, so the directories are printed one per line. In Figure 1.5 the list contains `/usr/local/sbin`, `/usr/local/bin`, `/usr/sbin`, `/usr/bin`, `/sbin`, `/bin`, `/usr/games`, `/usr/local/games` and `/snap/bin` (twice), which gives 10 lines and makes it easy to count the directories.

nosuchprog ; echo "exit code: $?" – runs a program that does not exist and then prints the special variable `$?`, which holds the exit code of the last command. The shell answers `command not found` and the printed code is 127, the standard code for a command that could not be located in any PATH directory.

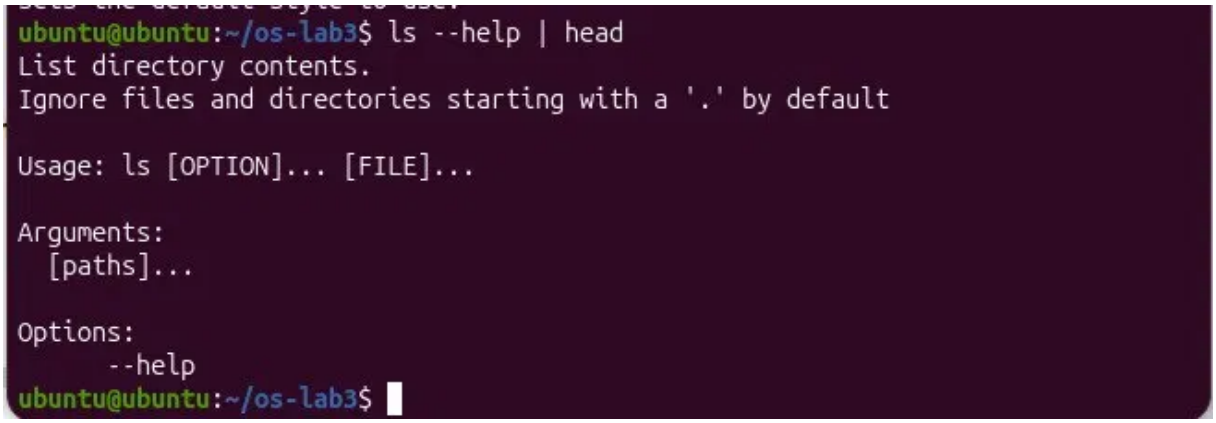


Figure 1.6: Output of ls –help | head

ls –help | head – prints the short built-in help of `ls` and pipes it to `head`, so only the first ten lines are shown. Figure 1.6 shows the description of the program, its usage line `ls [OPTION]... [FILE]...` and the beginning of the options list. This is a fast way to remember an option without opening the manual.

man ls – opens the manual page of `ls`. Inside the pager the user can search with `/sort`, jump to the next match with `n` and leave with `q`. The search shows that the option `-t` sorts the output by modification time, which answers the last question of this part.

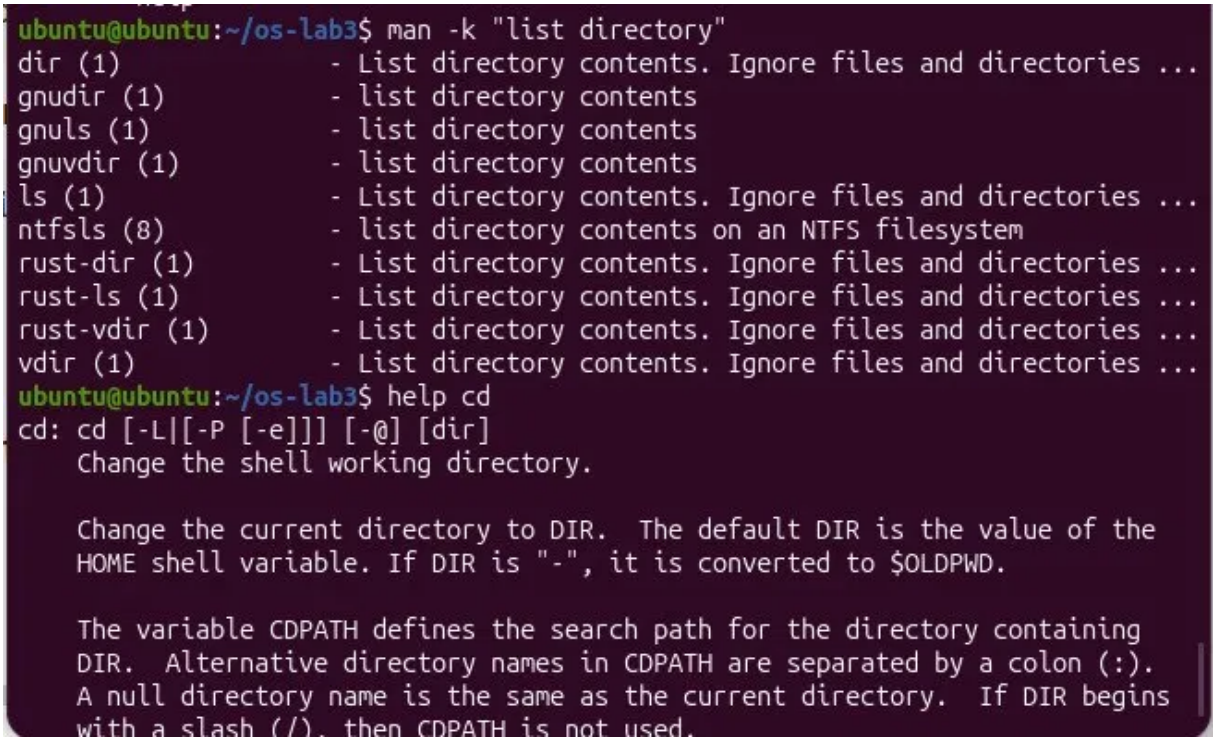


Figure 1.7: Output of man -k "list directory" and help cd

man -k "list directory" – searches the short descriptions of all manual pages for the given phrase. Figure 1.7 shows several matching programs such as `dir`, `ls`, `vdir` and

`ntfsls`, all described as tools that list directory contents. This is useful when the name of a command is unknown, but its purpose is known.

help cd – shows the help text of a shell built-in. Built-ins have no separate manual page, so `help` is used instead of `man`. The output explains that `cd` changes the shell working directory, that the default directory is the value of `HOME`, and that `cd -` returns to `$OLDPWD`.

Part 2: What the shell does to your line

Questions:

- For each line: your prediction, the real output, and one sentence if they differ (the table).
- echo knows nothing about files, yet echo *.c printed file names. Who produced them?
- Why did ls alfa beta.txt fail? Give two different ways to fix it.
- What is the difference between single and double quotes? Explain why ls may show the file with a space as 'alfa beta.txt'.

Answers:

- The predictions, the real outputs and the explanations are given in Table 1.
- The names were produced by the shell. Before `echo` starts, bash expands the pattern `*.c` (globbing) into the list of matching file names `a.c b.c` and passes that list to `echo` as ordinary arguments. `echo` only prints what it receives.
- Without quotes, the space separates the words, so `ls` received two arguments, `alfa` and `beta.txt`, and neither exists. It can be fixed by quoting the name, `ls "alfa beta.txt"`, or by escaping the space with a backslash, `ls alfa\ beta.txt`.
- Single quotes keep every character literally, so no expansion takes place. Double quotes protect spaces and wildcards, but still allow expansion of variables (`$HOME`), command substitution (`$(...)`) and arithmetic. `ls` prints the name in quotes when it writes to a terminal and the name contains special characters such as a space, so that the user can copy it and use it safely in another command.

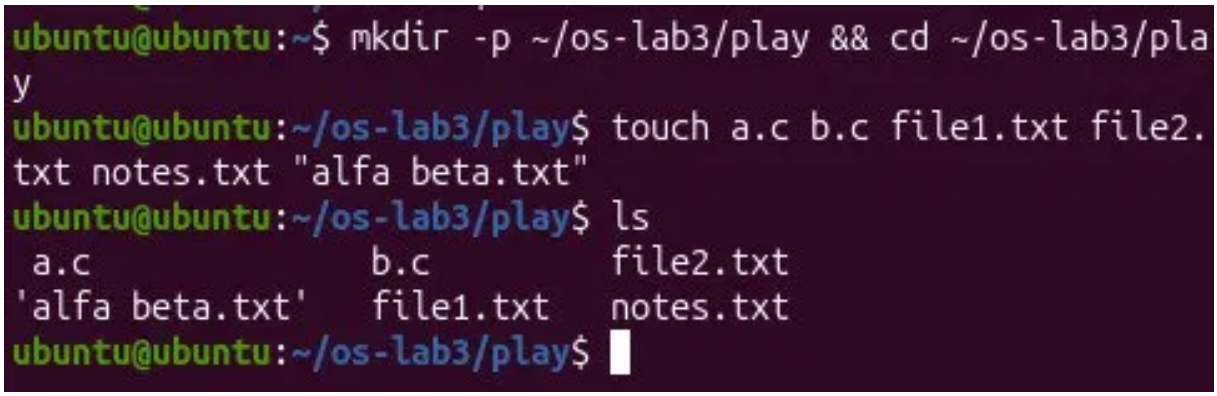


Figure 2.1: Files created in the play directory

mkdir -p ~/os-lab3/play && cd ~/os-lab3/play – creates the test directory and enters it only if the creation succeeded (the operator `&&` runs the second command only after a successful first one).

touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt" – creates six empty files. The last name is quoted, so it is created as one file whose name contains a space. The following `ls` in Figure 2.1 lists all six files and displays `alfa beta.txt` between single quotes. These files are the test material for the globbing experiments.



Figure 2.2: Commands results

Command	My prediction	Real output	Explanation (if different)
`echo *.c`	`a.c b.c`	`a.c b.c`	–
`echo file?.txt`	`file1.txt file2.txt`	`file1.txt file2.txt`	–
`echo [a-b]*`	`a.c b.c`	`a.c alfa beta.txt b.c`	The file `alfa beta.txt` also starts with `a`, so it matches; the names are sorted alphabetically.
`echo *.xyz`	empty line	`*.xyz`	No file matches, so bash leaves the pattern unchanged and `echo` prints it.
`ls alfa beta.txt`	error	two errors: cannot access `alfa` and `beta.txt`	– (the space splits the name into two arguments).
`ls "alfa beta.txt"`	`alfa beta.txt`	`'alfa beta.txt'`	Same file; `ls` adds quotes only for display.
`echo '$HOME' "$HOME" \$HOME`	`$HOME /home/ubuntu $HOME`	`$HOME /home/ubuntu //home/ubuntu`	In the executed line the last word was typed as `/$HOME` instead of `\$HOME`, so the variable was expanded after a slash. With the backslash the output is `$HOME`.
`echo {1..5} ; echo $((7 * 6))`	`1 2 3 4 5` and `42`	`1 2 3 4 5` and `42`	–
`echo "today is $(date +%A), I am $(whoami)"`	`today is Friday, I am ubuntu`	`today is Friday , I am ubuntu`	The day name comes from the clock of the virtual machine, and a space was typed before the comma.
`touch log{1..3}.txt && ls log*`	`log1.txt log2.txt log3.txt`	`log1.txt log2.txt log3.txt`	–

Table 1: Predictions compared with the real output

echo *.c – the shell replaces `*.c` with the names of all files ending in `.c`, so the output in Figure 2.2 is `a.c b.c`. The asterisk stands for any sequence of characters.

echo file?.txt – the question mark matches exactly one character, so the pattern matches `file1.txt` and `file2.txt`, but not `notes.txt`.

echo [a-b]* – the bracket expression matches one character from the range `a` to `b`, followed by anything. The result contains `a.c`, `alfa beta.txt` and `b.c`. The name with the space is printed as one item, because it comes from the expansion and is not split again.

echo *.xyz – when no file matches the pattern, the shell does not report an error and passes the pattern unchanged to `echo`. The output `*.xyz` in Figure 2.2 proves that the expansion happens in the shell, not in the program.

ls alfa beta.txt – since the space separates arguments, `ls` looks for two files named `alfa` and `beta.txt` and prints two errors `No such file or directory` (visible in the lower part of Figure 2.2, which also contains the fix).

ls "alfa beta.txt" – the double quotes keep the space inside one argument, so `ls` finds the file and prints `'alfa beta.txt'`. The same result is obtained with the backslash form `alfa\ beta.txt`.

echo '$HOME' "$HOME" \$HOME – compares three ways of protecting the dollar sign. Single quotes print the text `$HOME` literally, double quotes allow the variable to be expanded to `/home/ubuntu`, and a backslash protects only the next character. As shown in Figure 2.2, the third word was typed with a slash, so it printed `//home/ubuntu`, which confirms that an unprotected variable is always expanded.

echo {1..5} ; echo $((7 * 6)) – brace expansion generates the sequence `1 2 3 4 5`, and arithmetic expansion computes `42`. Both are done by the shell before `echo` runs.

echo "today is $(date +%A), I am $(whoami)" – command substitution runs `date` and `whoami` and inserts their output into the string. The result `today is Friday , I am ubuntu` shows the day name and the user name of the machine.

touch log{1..3}.txt && ls log* – brace expansion creates `log1.txt`, `log2.txt` and `log3.txt` with a single command, and the glob `log*` lists them. This shows how expansions can be combined.

Part 3: Redirection, chaining and jobs

Questions:

- Which file got the error message, and which got the listing?
- What is the difference between > all.txt 2>&1 and 2>&1 > all.txt? Try both and explain why the result differs.
- Why does wc -l < access.log print only a number, while wc -l access.log also prints the file name?
- Explain each operator in one sentence: ; && ||.

Answers:

- The error message `ls: cannot access '/nope': No such file or directory` went to `err.txt` (standard error, descriptor 2), and the listing of `/etc` (249 lines) went to `out.txt` (standard output, descriptor 1).

- The shell processes redirections from left to right. In `> all.txt 2>&1` the standard output is first sent to the file and then the standard error is duplicated to the place where standard output points now, so both streams end up in `all.txt` and nothing appears on the screen. In `2>&1 > all.txt` the standard error is first duplicated to the current standard output (the terminal) and only after that the standard output is moved to the file, so the error is still printed on the screen and only the listing is written to `all.txt`.
- With `< access.log` the shell opens the file and gives it to `wc` as standard input, so `wc` does not know any file name and prints only the number. With `wc -l access.log` the file name is an argument, so `wc` prints it after the number.
- The operator `;` runs the next command regardless of the result of the previous one; `&&` runs the next command only if the previous one succeeded (exit code 0); `||` runs the next command only if the previous one failed (non-zero exit code).

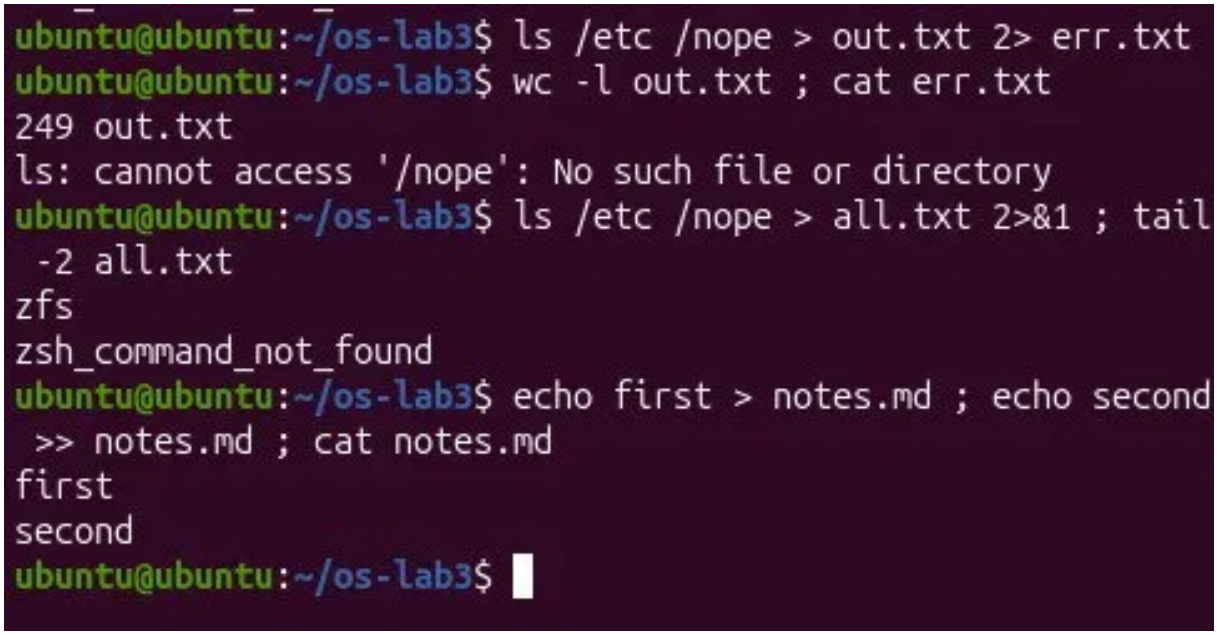


Figure 3.1: stdout and stderr in separate files

ls /etc /nope > out.txt 2> err.txt – lists two paths, where `/nope` does not exist. The standard output is written to `out.txt` and the standard error to `err.txt`, so nothing is printed on the terminal (Figure 3.1).

wc -l out.txt ; cat err.txt – counts the lines of the listing, which gives `249 out.txt`, and prints the content of the error file, `ls: cannot access '/nope': No such file or directory`. This proves that the two streams are independent.

ls /etc /nope > all.txt 2>&1 ; tail -2 all.txt – sends both streams to the same file. The last two lines, `zfs` and `zsh_command_not_found`, belong to the listing of `/etc`, and the error message is stored in the same file, at the beginning, because `ls` reports it before printing the directory content.

echo first > notes.md ; echo second » notes.md ; cat notes.md – the operator `>` creates or overwrites a file, while `»` appends to it. The output `first` and `second` shows that the second command did not erase the first line.

```
ubuntu@ubuntu:~/os-lab3$ echo first > notes.md ; echo second
 >> notes.md ; cat notes.md
first
second
ubuntu@ubuntu:~/os-lab3$ wc -l < access.log
500
ubuntu@ubuntu:~/os-lab3$ ls /nope 2> /dev/null || echo "fail
ed with code $?"
failed with code 2
```

Figure 3.2: Error messages and result of running commands

wc -l < access.log – the input redirection feeds the file to `wc`, which prints only the number of lines (500), without the file name, as explained in the answers above.

ls /nope 2> /dev/null || echo "failed with code $?" – the error message is discarded by sending it to the special file `/dev/null`. Because `ls` fails, the operator `||` runs the `echo` command, which prints the exit code of the failed command (2).

```
ubuntu@ubuntu:~/os-lab3$ mkdir -p tmp && cd tmp && echo "I a
m in $(pwd)" ; cd ~/os-lab3
I am in /home/ubuntu/os-lab3/tmp
```

Figure 3.3: The ; and && operators

mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3 – each command in the chain runs only if the previous one succeeded. The printed text `I am in /home/ubuntu/os-lab3/tmp` in Figure 3.3 shows that the directory was created and entered. The final `;` runs `cd ~/os-lab3` in any case, so the work returns to the lab directory.

```
ubuntu@ubuntu:~/os-lab3$ sleep 100 &
[1] 29476
ubuntu@ubuntu:~/os-lab3$ jobs ; kill %1 ; jobs
[1]+  Running                 sleep 100 &
[1]+  Running                 sleep 100 &
[1]+  Terminated              sleep 100
ubuntu@ubuntu:~/os-lab3$
```

Figure 3.4: Running a background job and stopping it

sleep 100 & – the ampersand starts the command in the background. The shell prints the job number `[1]` and the PID `29476` and immediately returns the prompt.

jobs ; kill %1 ; jobs – the first `jobs` lists the job as `Running`. `kill %1` sends a termination signal to job number 1. In Figure 3.4 the second `jobs` still shows `Running` for a moment, because the shell has not yet updated the status, and then the shell reports `Terminated`, which confirms that the background process was stopped.

Part 4: Answer questions with one-liners

Questions:

- Q1. How many requests failed?
- Q2. How many different pages were requested, and which?
- Q3. How many requests did each page get?
- Q4. Which user or users have the most failed requests, and how many?
- Q5. How many requests did user3 make, and how many of them failed?
- Q6. Print the last 3 failed requests, showing only time and user.
- Q7. Which login shells appear in /etc/passwd, and how many accounts use each? (field 7, separator :)
- Q8. PREDICT, then explain: why do ls /etc | wc -l and ls -l /etc | wc -l differ by exactly one?

Answers:

- Q1. 100 requests failed.
- Q2. Four different pages were requested: `/page0`, `/page1`, `/page2` and `/page3`.
- Q3. Each page received 125 requests, and $4 \times 125 = 500$, which matches the number of lines in the log.
- Q4. The users with the most failed requests are user3 and user5, with 15 failures each; the other users have 14 failures each ($2 \times 15 + 5 \times 14 = 100$, which matches Q1).
- Q5. User3 made 72 requests, and 15 of them failed.
- Q6. The last three failed requests are `10:10 user0`, `10:15 user5` and `10:20 user3`.
- Q7. The shells are `/usr/sbin/nologin` (40 accounts), `/bin/false` (5), `/bin/bash` (2), `/bin/sync` (1) and `/usr/bin/subiquity-shell` (1).
- Q8. The prediction is that the long format adds a header line. Indeed, `ls -l` prints an extra first line `total ...` with the number of disk blocks, which is not a file, so its count is exactly one larger.

```
ubuntu@ubuntu:~/os-lab3$ for i in $(seq 1 500); do echo "2026-10-05 10:$(printf
%02d $((i % 60))) user$((i % 7)) $([ $(( i %
5)) -eq 0 ] && echo FAIL || echo OK) /page$(i % 4))"
> done > access.log
```

Figure 4.1: Generating the access.log data set

for i in $(seq 1 500); do echo "..."; done > access.log – the loop runs 500 times and writes one line for each value of `i`. Every line contains the date, a minute computed with `i % 60`, the user (`user$((i % 7))`), the result (`FAIL` for every fifth request and `OK` otherwise) and one of four pages (`/page$((i % 4))`). The output of the whole loop is redirected into the file `access.log`, which is the data set used in this part (Figure 4.1).

head -3 access.log ; wc -l access.log – prints the first three lines of the file and counts all its lines.

head -10 access.log – shows the first ten records. It makes the structure of a line visible: date (field 1), time (field 2), user (field 3), result (field 4) and page (field 5). This structure is used by the `cut` commands that follow, with the separator `-d' '`.

```
 GNU nano 8.7.1                    oneliners.sh *
# Q1. How many requests failed?S
grep FAIL access.log | wc -l

# Q2. How many different pages were requested, and which?
cut -d' ' -f5 access.log | sort | uniq

# Q3. How many requests did each page get?
cut -d' ' -f5 access.log | sort | uniq -c

# Q4. Which user or users have the most failed requests, and how many?
grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn

# Q5. How many requests did user3 make, and how many of them failed?
echo "total=$(grep ' user3 ' access.log | wc -l) failed=$(grep ' user3 ' access>

# Q6. Print the last 3 failed requests, showing only time and user.
grep FAIL access.log | tail -3 | cut -d' ' -f2,3

# Q7. Which login shells appear in /etc/passwd, and how many accounts use each?
cut -d':' -f7 /etc/passwd | sort | uniq -c
```

Figure 4.2: contents of the file oneliners.sh

Q1: grep -c FAIL access.log – counts the lines that contain the word `FAIL`. The result is 100, which is exactly one request in five.

Q2: cut -d' ' -f5 access.log | sort -u – extracts the fifth field (the page), sorts the values and removes duplicates. Four different pages remain: `/page0`, `/page1`, `/page2` and `/page3`.

Q3: cut -d' ' -f5 access.log | sort | uniq -c – the same extraction, but `uniq -c` counts how many times each page appears. Every page was requested 125 times, and the sum is 500.

Q4: grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn – selects only the failed requests, keeps the user field, counts the failures per user and sorts the counts in descending numeric order. The first lines are `15 user5` and `15 user3`, so these two users have the most failures; the sum of all counts is 100, as required by the self-check.

Q5: echo "total=$(grep -c user3 access.log) failed=$(grep user3 access.log | grep -c FAIL)" – counts all lines of `user3` and, with a second filter, only those that also contain `FAIL`. The output is `total=72 failed=15`.

Q6: grep FAIL access.log | tail -3 | cut -d' ' -f2,3 – takes the last three failed requests and prints only the time and user fields: `10:10 user0`, `10:15 user5` and `10:20 user3`.

Q7: cut -d: -f7 /etc/passwd | sort | uniq -c – the file `/etc/passwd` uses a colon as separator, and field 7 is the login shell. After sorting and counting, the result shows that most accounts use `/usr/sbin/nologin` (40), which means that they are service accounts without an interactive login.

Q8: echo "ls -l adds one extra line ('total ...')" – this line of `oneliners.sh` prints the explanation of the difference between `ls /etc | wc -l` and `ls -l /etc | wc -l`. The two counts differ by exactly one because of the `total` line of the long format.

Bonus: grep -E '10:[0-5]0 .* FAIL' access.log | wc -l – the extended regular expression selects the lines where the minute ends in 0 and the request failed. There are 50 such lines.

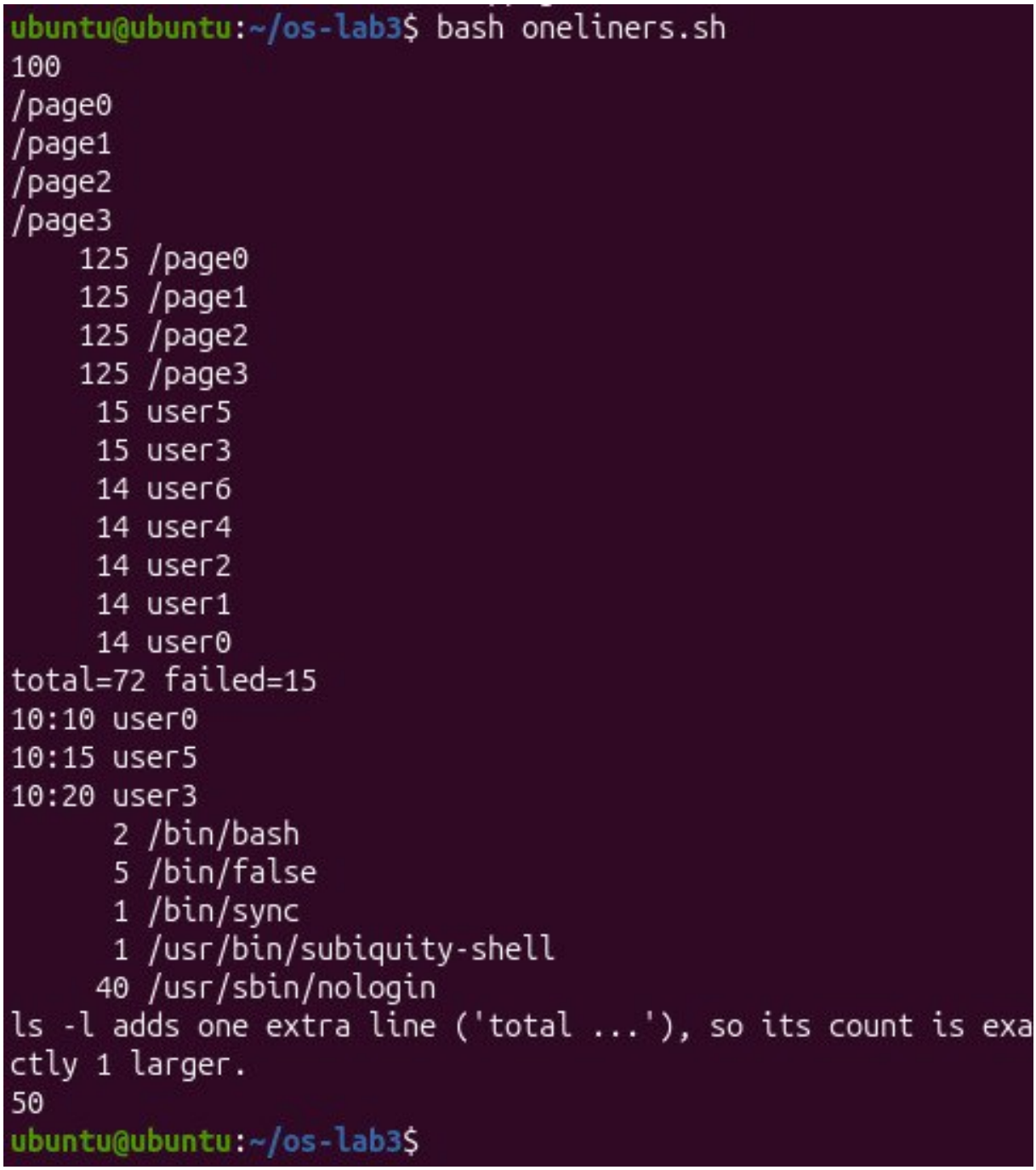


Figure 4.3: Full run of oneliners.sh

bash oneliners.sh – runs the file that contains all the one-line commands, each preceded by its question as a comment. Figure 4.3 shows the answers in order: 100 failed requests, the four pages, 125 requests for each page, the failures per user, `total=72 failed=15` for user3, the last three failures, the login shells with their counts, the explanation for Q8 and the bonus result 50. The totals agree with the self-check rules of the task (Q3 adds up to 500 and the failures per user add up to 100).

Part 5: Your first script

Questions:

- What does the first line #!/bin/bash do, and what happens if you run ./report.sh before chmod +x?
- Why must you type ./report.sh and not just report.sh?
- Why does the script write the error with >&2 and end with exit 1?

Answers:

- The first line, called shebang, tells the kernel which interpreter must execute the file, here `/bin/bash`. If the script is run before `chmod +x`, the shell refuses to run it and prints `Permission denied`, because the file has no execute permission.
- The current directory is not in the `PATH`, so typing only `report.sh` makes the shell search for it in the PATH directories and answer `command not found`. The prefix `./` gives an explicit path, which means the file in the current directory.
- The redirection `>&2` sends the error message to standard error, so it is separated from the normal output and is not mixed with data that may be redirected or piped. The command `exit 1` ends the script with a non-zero code, which tells the caller (another script, `&&`, `||` or `$?`) that the script failed.



Figure 5.1: Contents of report.sh

nano report.sh – the script was written in the `nano` editor. Figure 5.1 shows the shebang line, the usage comment, the check `if [ ! -f "$log" ]` that verifies the existence of the file, the computation of `total` with `wc -l < "$log"` and of `fails` with `grep -c FAIL "$log"`, and the completed TODO: the loop `for i in $(seq 0 6)`, which counts for every user the lines that contain both `user$i` and `FAIL` and prints them as `userN: X failures`.

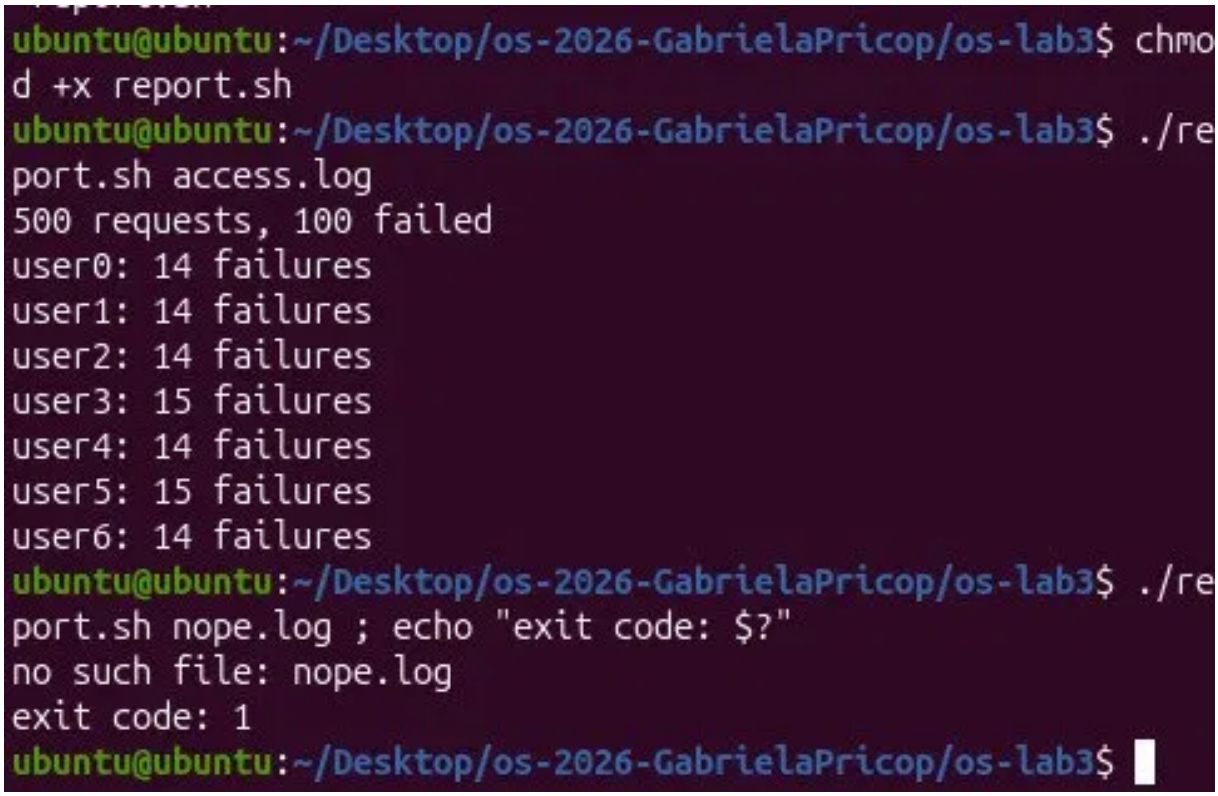


Figure 5.2: Running report.sh on access.log and error handling for a missing file

chmod +x report.sh – adds the execute permission to the script, so the system can run it as a program.

./report.sh access.log – runs the script with the log as argument. As shown in Figure 5.2, the first line is `500 requests, 100 failed`, followed by the failures per user from `user0` to `user6` (14 failures for each user, except user3 and user5, which have 15). The values are identical to the results obtained with the one-liners.

./report.sh nope.log ; echo "exit code: $?" – runs the script with a file that does not exist. The script prints `no such file: nope.log` on standard error and stops with code 1, which is confirmed by the line `exit code: 1` in Figure 5.2. This proves that the error handling works.

Conclusion

The laboratory work introduced the Linux command line and the way in which the bash shell processes what the user types. The first part showed how Tab completion and the history search speed up the work, how `type` and `which` distinguish built-ins from external programs, why `cd` must be a built-in, and how the `PATH` variable, the exit code 127 and the help tools (`man`, `-help`, `man -k`, `help`) allow the user to find information without a browser. The second part demonstrated that the shell, not the program, expands wildcards, braces, variables, arithmetic and command substitutions, and that single quotes, double quotes and backslashes control this process, which explained the error produced by a file name with a space. The third part presented the standard streams, the redirection operators `>`, `»`, `<` and `2>&1`, the importance of the order of redirections, the chaining operators `;`, `&&` and `||`, and the management of background jobs with `&`, `jobs` and `kill`.

The last two parts applied these ideas to real data. With filters such as `grep`, `cut`, `sort`, `uniq`, `wc`, `head` and `tail`, joined by pipes, eight questions about an artificial access log with 500 lines and about the file `/etc/passwd` were answered with one-line commands, and the results were verified with simple consistency checks. Finally, the most useful commands were turned into the script `report.sh`, which uses a shebang, a condition, a loop, command substitution, error output on standard error and a meaningful exit code. Overall, the laboratory work gave practical skills in predicting the behaviour of the shell, controlling input and output, and automating small data analysis tasks, and all the objectives of the work were achieved.

References

1. *GNU Bash Reference Manual* `https://www.gnu.org/software/bash/manual/bash.html`
2. *Linux man-pages project* `https://man7.org/linux/man-pages/`
3. *William Shotts. The Linux Command Line: A Complete Introduction, 2nd Edition. No Starch Press, 2019*
4. *Cameron Newham, Bill Rosenblatt. Learning the bash Shell, 3rd Edition. O'Reilly Media, 2005*